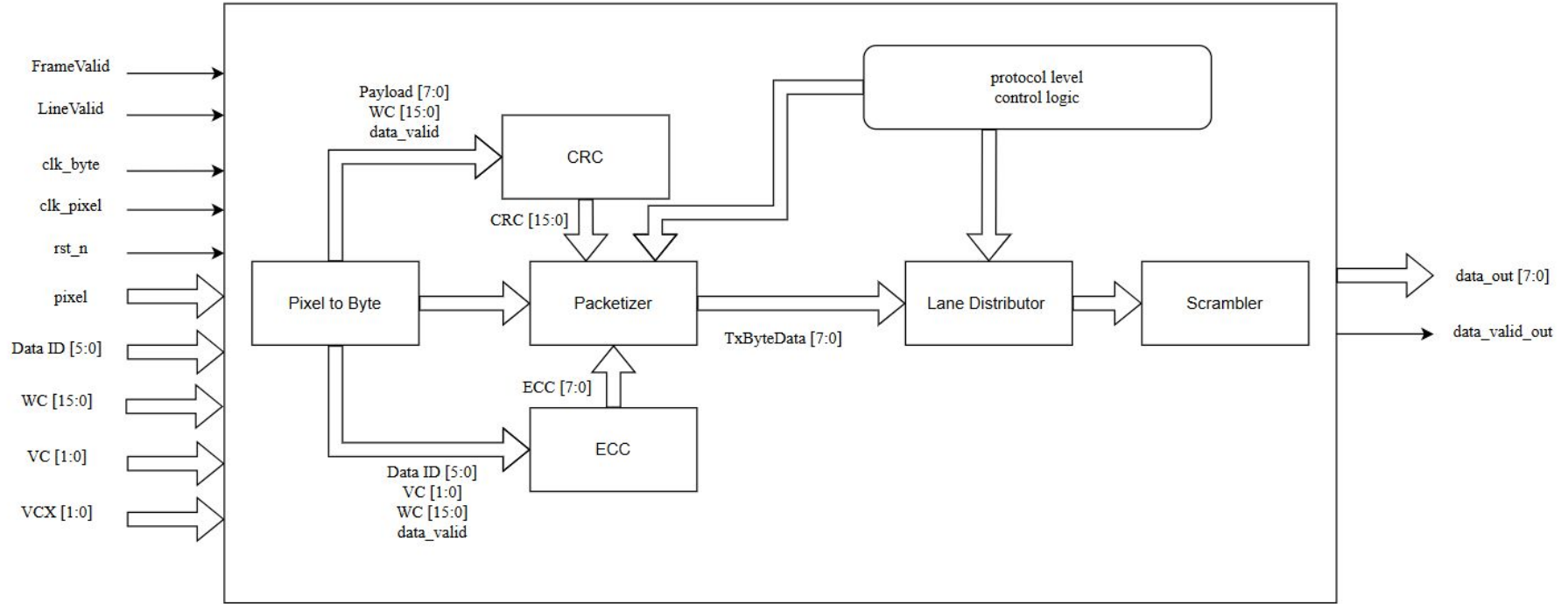


Design and Implementation of CSI Controller IP

CSI-2 Transmitter



Pixel to Byte block

Pixel to Byte converter-Pixel Domain

I-RSTN	Active-low reset .
I_PIXEL_CLK	Pixel clock — the clock domain in which pixel data is sampled (pixel domain)
I_FV	Frame valid — indicates that pixel data belongs to the active frame
I_LV	Line valid — indicates which pixel data in a line are valid (vs blanking)
I_PIXEL[...]	The parallel pixel data bus. Depending on the pixel format.
I_DT[5:0]	Data Type field (6 bits) from the CSI-2 header — tells which kind of data is coming (RAW, RGB, etc.).
I_WC[15:0]	Word Count (16 bits) from the header of the incoming packet — how many payload bytes follow.

Pixel to Byte converter-Byte Domain

O_DATA_EN	Byte data valid indicator, active-high
O_DATA	The converted byte-level data output (after packing from pixel domain), and the width is dependent on the lanes (1-4) and data modes (1:8/1:16).
O_FV_START	Pulse marking the start of a frame in the byte-domain output (converted from fv_i / vsync)
O_LV_END	Pulse marking the start of a line in the output domain
O_LV_END	Pulse marking the end of a line in output domain
O_FV_END	Pulse marking the end of a frame in the byte-domain
I_BYTE_CLK	Byte clock — the clock domain in which pixel data is sampled (pixel domain)

ECC

ECC Signal Flow: Transmitter (TX) Side - Generation

Function: On the transmitter side, the ECC block's function is **ECC Generation**. It takes the raw 26-bit header data and calculates the 6-bit ECC code.

Input from Protocol Layer:

- **header_data_in[25:0]**: The 26-bit header containing Data ID, Word Count, and VCX. This is the data to be protected.
- **header_valid_in**: A control signal indicating that **header_data_in** is valid and ready.
- **clk, rst**: Standard clock and reset signals.

Internal Process:

- The **header_data_in** feeds into parallel XOR logic (as defined by the Parity-Check Matrix).
- This logic calculates the 6 individual ECC bits simultaneously.

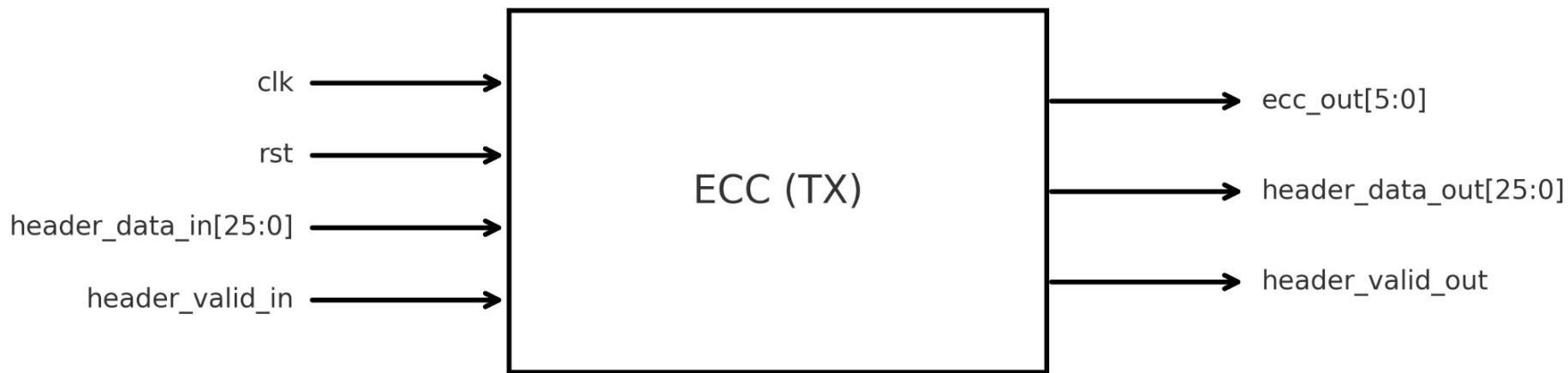
Output to Packetizer:

- **ecc_out[5:0]**: The newly calculated 6-bit ECC code.
- **header_data_out[25:0]**: The original 26-bit header data, passed through.
- **header_valid_out**: Indicates that both outputs are valid.

Next Step: The **Packetizer** module combines **header_data_out** and **ecc_out** to form the final 32-bit packet header for transmission via the D-PHY physical layer.

ECC Signal Flow: Transmitter (TX) Side - Generation

ECC Signal Flow — TX (Generation)



Example: ECC Generation at the TX

Scenario: We need to transmit a packet containing a line of RAW10 image data, 1280 pixels long, on Virtual Channel 1.

1. **Assemble Header Data (Inputs to ECC Block):**

- **Data Type (DT):** For RAW10, the code is 0x2B (binary 101011).
- **Virtual Channel (VC):** Channel 1 is binary 01.
- **Word Count (WC):** For RAW10, 4 pixels are packed into 5 bytes. So, 1280 pixels = $(1280 / 4) * 5 = 1600$ bytes. The WC is 1600, which is 0x0640 in hex.
- **Virtual Channel Extension (VCX):** We'll use 0, which is binary 00.
- The 26-bit **header_data_in** signal is assembled as {VCX, WC[15:0], VC, DT}: {00, 00000110, 01000000, 01, 101011}

2. **Internal Process: Calculate ECC:**

- The ECC Generator takes these 26 bits (D0 to D25) and applies the XOR rules from (**ECC Parity Generation Rules**) of the MIPI specification.
- For example, to calculate the first parity bit, **P0**:
 $P0 = D0 \oplus D1 \oplus D2 \oplus D4 \oplus D5 \dots$ and so on for all bits marked in the P0 column.
- This happens for all 6 parity bits in parallel.

3. **Final Output:**

- For this specific header, the calculated 6-bit **ecc_out** value would be 0x3A (binary 111010).
- The ECC block then outputs this value alongside the original header data to the **Packetizer**, which assembles the final 32-bit word for transmission: {VCX, WC[15:0], VC, DT, ECC} {00, 00000110, 01000000, 01101011, 111010} (Note: VC and DT are combined into the 8-bit Data ID)

ECC Parity Generation Rules

Bit	P7	P6	P5	P4	P3	P2	P1	P0	Hex
0	0	0	0	0	0	1	1	1	0x07
1	0	0	0	0	1	0	1	1	0x0B
2	0	0	0	0	1	1	0	1	0x0D
3	0	0	0	0	1	1	1	0	0x0E
4	0	0	0	1	0	0	1	1	0x13
5	0	0	0	1	0	1	0	1	0x15
6	0	0	0	1	0	1	1	0	0x16
7	0	0	0	1	1	0	0	1	0x19
8	0	0	0	1	1	0	1	0	0x1A
9	0	0	0	1	1	1	0	0	0x1C
10	0	0	1	0	0	0	1	1	0x23
11	0	0	1	0	0	1	0	1	0x25
12	0	0	1	0	0	1	1	0	0x26
13	0	0	1	0	1	0	0	1	0x29
14	0	0	1	0	1	0	1	0	0x2A
15	0	0	1	0	1	1	0	0	0x2C
16	0	0	1	1	0	0	0	1	0x31
17	0	0	1	1	0	0	1	0	0x32
18	0	0	1	1	0	1	0	0	0x34
19	0	0	1	1	1	0	0	0	0x38
20	0	0	0	1	1	1	1	1	0x1F
21	0	0	1	0	1	1	1	1	0x2F
22	0	0	1	1	0	1	1	1	0x37
23	0	0	1	1	1	0	1	1	0x3B
24	0	0	1	1	1	1	0	1	0x3D
25	0	0	1	1	1	1	1	0	0x3E

CRC

Checksum Generation

- ❑ Checksum is used to detect possible errors in transmission.
- ❑ The checksum is realized as 16-bit CRC (Cyclic Redundancy Check) based on the generator polynomial $x^{16}+x^{12}+x^5+x^0$.
- ❑ It is used for Long Packet payloads.
- ❑ The CRC shift register is initialized to **0xFFFF** at the beginning of each packet.

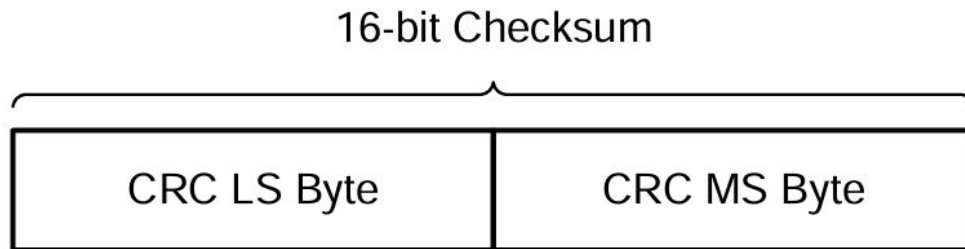
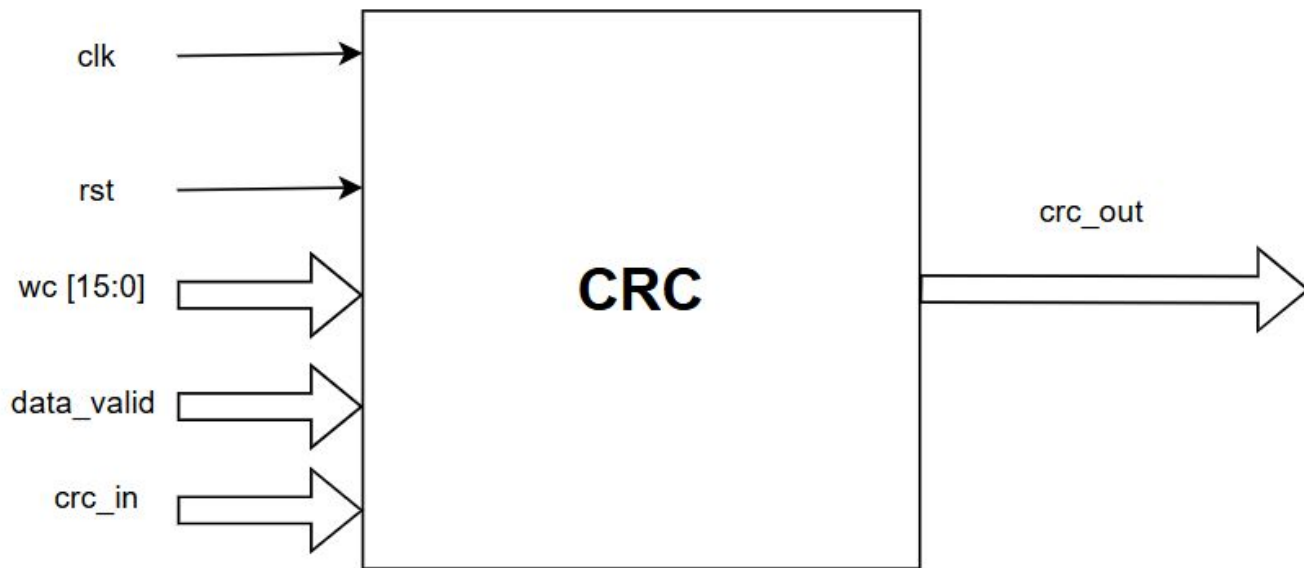


Figure 66 Checksum Transmission Byte Order

Cyclic Redundancy Check

At transmitter side:

CRC Calculation



Checksum Generation

Definition of 16-bit CRC Shift Register:

1 0001 0000 0010 0001 = 0x11021 **Polynomial: $x^{16} + x^{12} + x^5 + x^0$**

bit-reversed = 0x8408

CRC Computation Examples:

Input Data Bytes:

FF 00 00 02 B9 DC F3 72 BB D4 B8 5A C8 75 C2 7C 81 F8 05 DF FF 00 00 01

Checksum Generation

CRC Computation Examples:

Input Data Bytes:

FF 00 00 02 B9 DC F3 72 BB D4 B8 5A C8 75 C2 7C 81 F8 05 DF FF 00 00 01

Byte 0 = 0xFF = 1111 1111

Start: CRC = 0xFFFF

xor = (CRC & 1) XOR input_bit

- If xor == 1: CRC = (CRC >> 1) ^ POLY
- Else: CRC = CRC >> 1

Bit0 = 1: (1 XOR 1)=0, shift: CRC=0x7FFF

Bit1 = 1: shift: CRC=0x3FFF

Bit2 = 1: shift: CRC=0x1FFF

Bit3 = 1: shift: CRC=0x0FFF

Bit4 = 1: shift: CRC=0x07FF

Bit5 = 1: shift: CRC=0x03FF

Bit6 = 1: shift: CRC=0x01FF

Bit7 = 1: shift: CRC=0x00FF

Checksum Generation

CRC Computation Examples:

Input Data Bytes:

FF 00 00 02 B9 DC F3 72 BB D4 B8 5A C8 75 C2 7C 81 F8 05 DF FF 00 00 01

Byte 1 = 0x00 = 0000 0000

Start: CRC = 0x00FF

$\text{xor} = (\text{CRC} \& 1) \text{ XOR input_bit}$

- If $\text{xor} == 1$: $\text{CRC} = (\text{CRC} \gg 1) \wedge \text{POLY}$
- Else: $\text{CRC} = \text{CRC} \gg 1$

Bit0 = 0: (1 XOR 0)=1

$\text{CRC} = (\text{0x00FF} \gg 1) \wedge \text{0x8408} = \text{0x8477}$

Bit3 = 0: (1 XOR 0)=1

$\text{CRC} = (\text{0xE711} \gg 1) \wedge \text{0x8408} = \text{0xF780}$

Bit1 = 0: (1 XOR 0)=1

$\text{CRC} = (\text{0x8477} \gg 1) \wedge \text{0x8408} = \text{0xC633}$

Bit4 = 0: (0 XOR 0)=0 shift: $\text{CRC} = \text{0xF780} \gg 1 = \text{0x7BC0}$

Bit5 = 0: (0 XOR 0)=0 shift: $\text{CRC} = \text{0x7BC0} \gg 1 = \text{0x3DE0}$

Bit6 = 0: (0 XOR 0)=0 shift: $\text{CRC} = \text{0x3DE0} \gg 1 = \text{0x1EF0}$

Bit7 = 0: (0 XOR 0)=0 shift: $\text{CRC} = \text{0x1EF0} \gg 1 = \text{0x0F78}$

Bit2 = 0: (1 XOR 0)=1

$\text{CRC} = (\text{0xC633} \gg 1) \wedge \text{0x8408} = \text{0xE711}$

Checksum Generation

CRC Computation Examples:

Input Data Bytes:

FF 00 00 02 B9 DC F3 72 BB D4 B8 5A C8 75 C2 7C 81 F8 05 DF FF 00 00 01

Byte 2 = 0x00 = 0000 0000

Start: CRC = 0x0F78

xor = (CRC & 1) XOR input_bit

- If xor == 1: CRC = (CRC >> 1) ^ POLY
- Else: CRC = CRC >> 1

Bit0 = 0: (0 XOR 0)=0 shift:

CRC = 0x0F78 >> 1 = 0x07BC

Bit4 = 0 (1 XOR 0)=1

CRC = (0x80F7>>1) ^ 0x8408 = 0xC07A

Bit1 = 0: (0 XOR 0)=0 shift:

CRC = 0x07BC >> 1 = 0x03DE

Bit5 = 0 (0 XOR 0)=0 shift:

CRC = 0xC07A >> 1 = 0x603D

Bit2 = 0: (0 XOR 0)=0 shift:

CRC = 0x03DE >> 1 = 0x01EF

Bit6 = 0 (1 XOR 0)=1

CRC = (0x603D>>1) ^ 0x8408 = 0xF821

Bit3 = 0: (1 XOR 0)=1

CRC = (0x01EF>>1) ^ 0x8408 = 0x80F7

Bit7 = 0 (1 XOR 0)=1

CRC = (0xF821>>1) ^ 0x8408 = 0xFFC0

Checksum Generation

Checksum LS byte and MS byte:
F0 00

CRC Computation Examples:

Input Data Bytes:

FF 00 00 02 B9 DC F3 72 BB D4 B8 5A C8 75 C2 7C 81 F8 05 DF FF 00 00 01

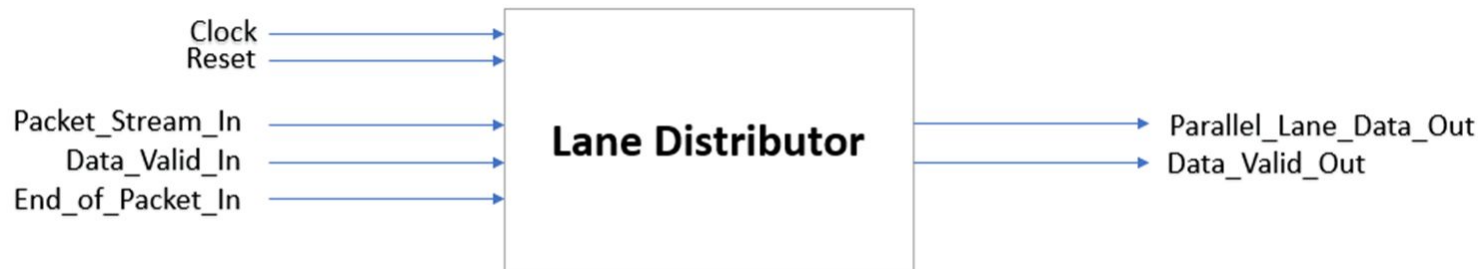
Byte	Input	CRC
0	FF	0x00FF
1	00	0x0F78
2	00	0xFFC0
3	02	0xE5E1
4	B9	0xDE28
5	DC	0xB175
6	F3	0xE18F
7	72	0x2C8B

Byte	Input	CRC
8	BB	0x31AF
9	D4	0xCD65
10	B8	0x0DA5
11	5A	0x0F75
12	C8	0x6E61
13	75	0x56CB
14	C2	0x9D97
15	7C	0x5940

Byte	Input	CRC
16	81	0xD7DC
17	F8	0x67F1
18	05	0xB1CC
19	DF	0x22AB
20	FF	0x1483
21	00	0xB687
22	00	0xF001
23	01	0x00F0

Lane Management

Lane Distributor



Lane Distributor is responsible for dividing the incoming serial data stream into multiple parallel lanes for efficient transmission. It processes the incoming packet data and distributes it across the available lanes.

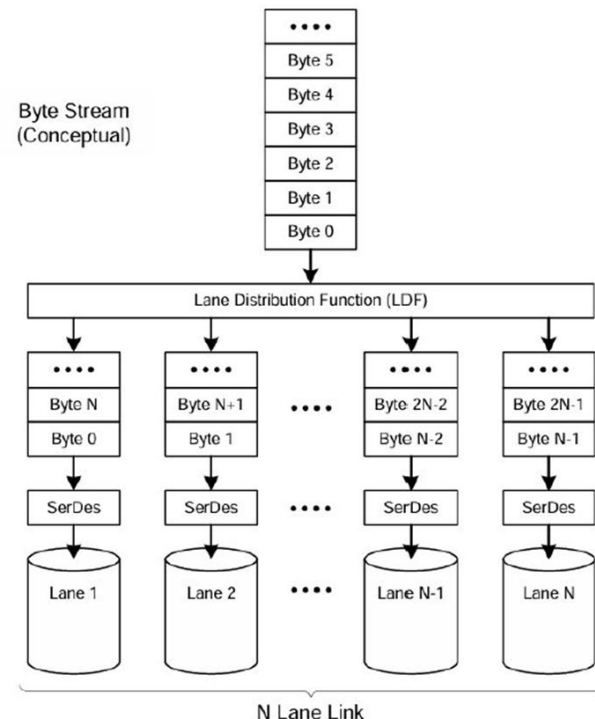
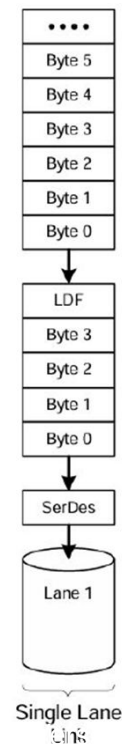
Lane Distributor

Short packets are typically small data packets that fit comfortably in a **single lane**. These packets may not need multiple lanes to carry the data.

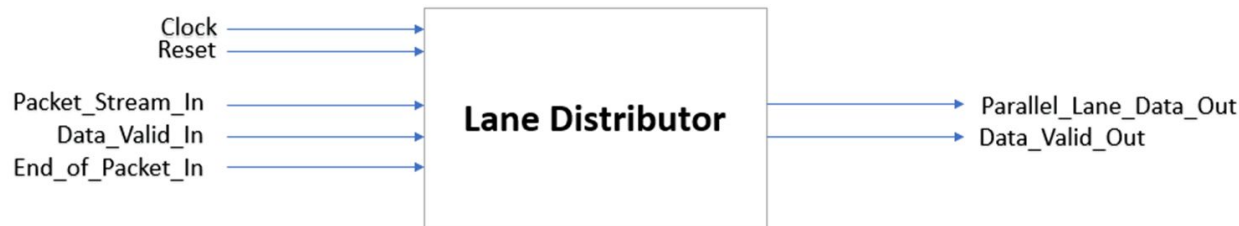
When the data is short, it can be sent in a **single lane** to optimize bandwidth and reduce complexity.

Long packets are larger data packets that require more bandwidth. In this case, the data is split across multiple lanes to speed up the transmission.

When the data is long, it is typically sent across **multiple lanes** to allow for parallel data transfer, thus improving performance.



Lane Distributor



Packet_Stream_In

The incoming serial data stream containing header, data, and footer.

The block receives this signal and begins processing the data, distributing it across parallel lanes.

Data_Valid_In

Indicates whether the incoming data stream (**Packet_Stream_In**) is valid.

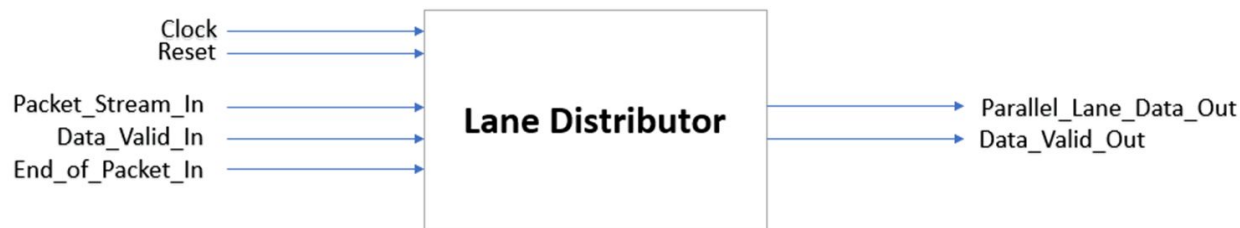
When this signal is high, the block processes the data. If it's low, the block ignores the incoming data.

End_of_Packet_In

Marks the end of the current data packet.

When this signal is activated, the block knows that the data packet is complete, and it can start processing the next packet.

Lane Distributor



Parallel_Lane_Data_Out

The distributed data that is now split across multiple lanes for parallel transmission.

The **Lane Distributor** divides the serial data into parallel chunks and sends them out through this signal.

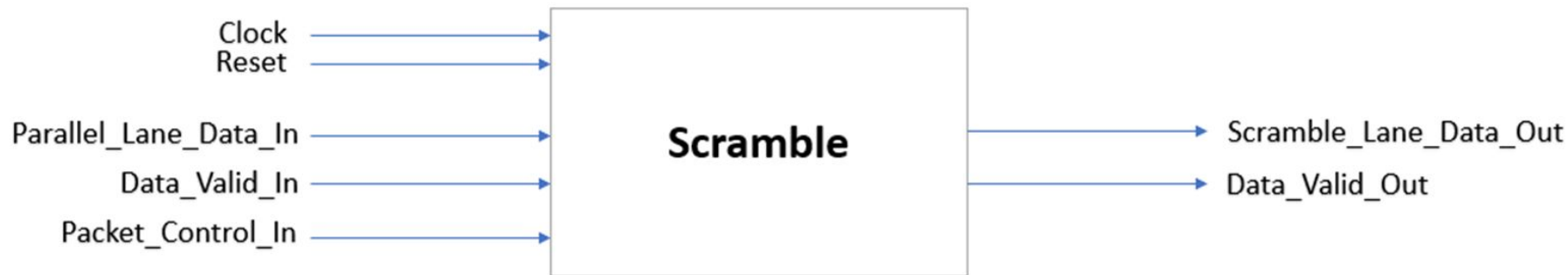
Data_Valid_Out

Confirms that the data distributed to the lanes is valid and ready for further processing.

When this signal is active, the downstream block can process the parallel data. It ensures the data integrity.

Scramble

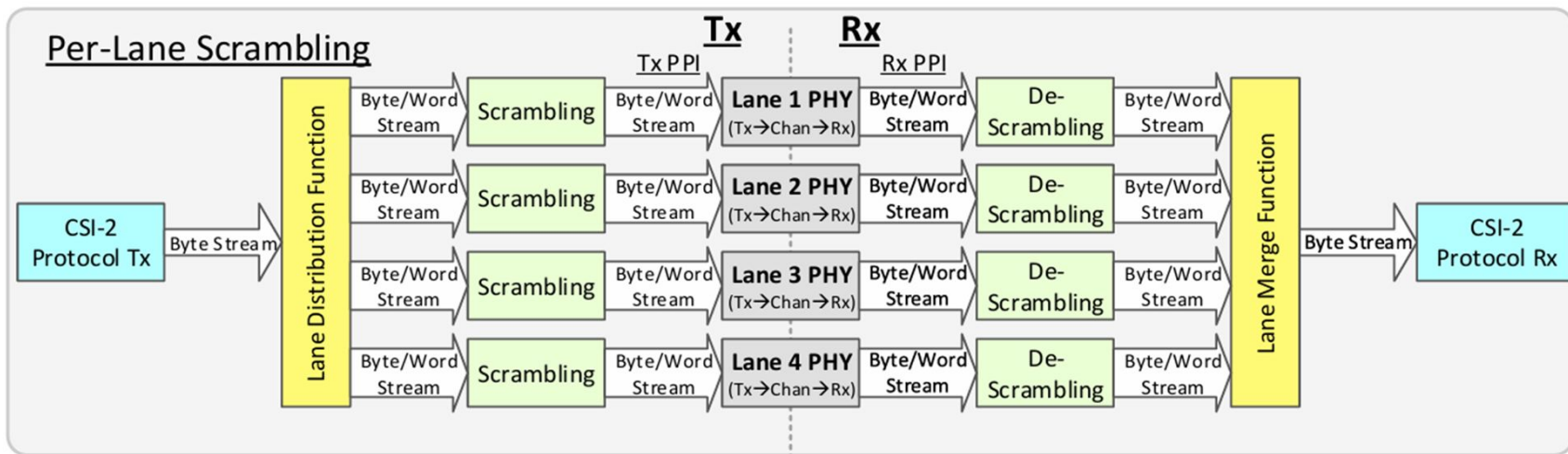
Scramble



The purpose of Data Scrambling is to mitigate the effects of EMI and RF self-interference by spreading the information transmission energy of the Link over a possibly large frequency band, using a data randomization technique.

Data Scrambling

Data scrambling is a technique used to randomize the transmitted data pattern without changing the actual information.



Data Scrambling EXAMPLE

Original Data (before scrambling):

If **original data** is an 16-bit sequence: 11001100 10101010

Generated PRBS (Pseudo-Random Bit Sequence):

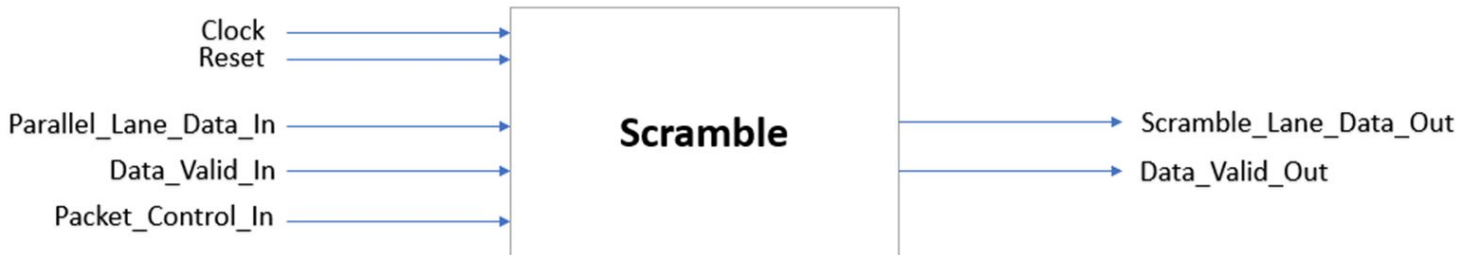
Let's assume the **PRBS** is: 10111010 01100101

The **PRBS** is generated from a **seed** using the **LFSR** (Linear Feedback Shift Register) method.

Scrambled Data (XOR Result): 01110110 11001111

Lane	Initial Seed Value
1	0x0810
2	0x0990
3	0x0a51
4	0x0bd0
5	0x0c30
6	0x0db0
7	0x0e70
8	0x0ff0

Scramble



Parallel_Lane_Data_In

The data that has been distributed across multiple lanes by the **Lane Distributor**.

The **Scramble** block receives the parallel data from the lanes and prepares it for scrambling.

Data_Valid_In

Indicates whether the incoming data (**Parallel_Lane_Data_In**) is valid and should be scrambled.

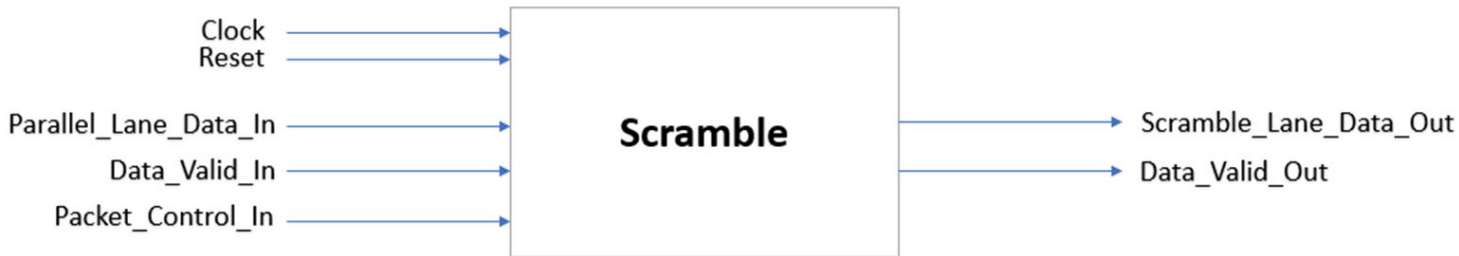
When **Data_Valid_In** is high, the **Scramble** block processes the incoming data. If it is low, the block ignores the data.

Packet_Control_In

Provides control information for scrambling.

This signal carries the control flags or metadata that guide how the scrambling process should occur (whether the data needs to be scrambled or not).

Scramble



Scramble_Lane_Data_Out

The scrambled data that is sent out of the **Scramble** block.

The data received through **Parallel_Lane_Data_In** is scrambled and then sent out through this signal for further processing or transmission.

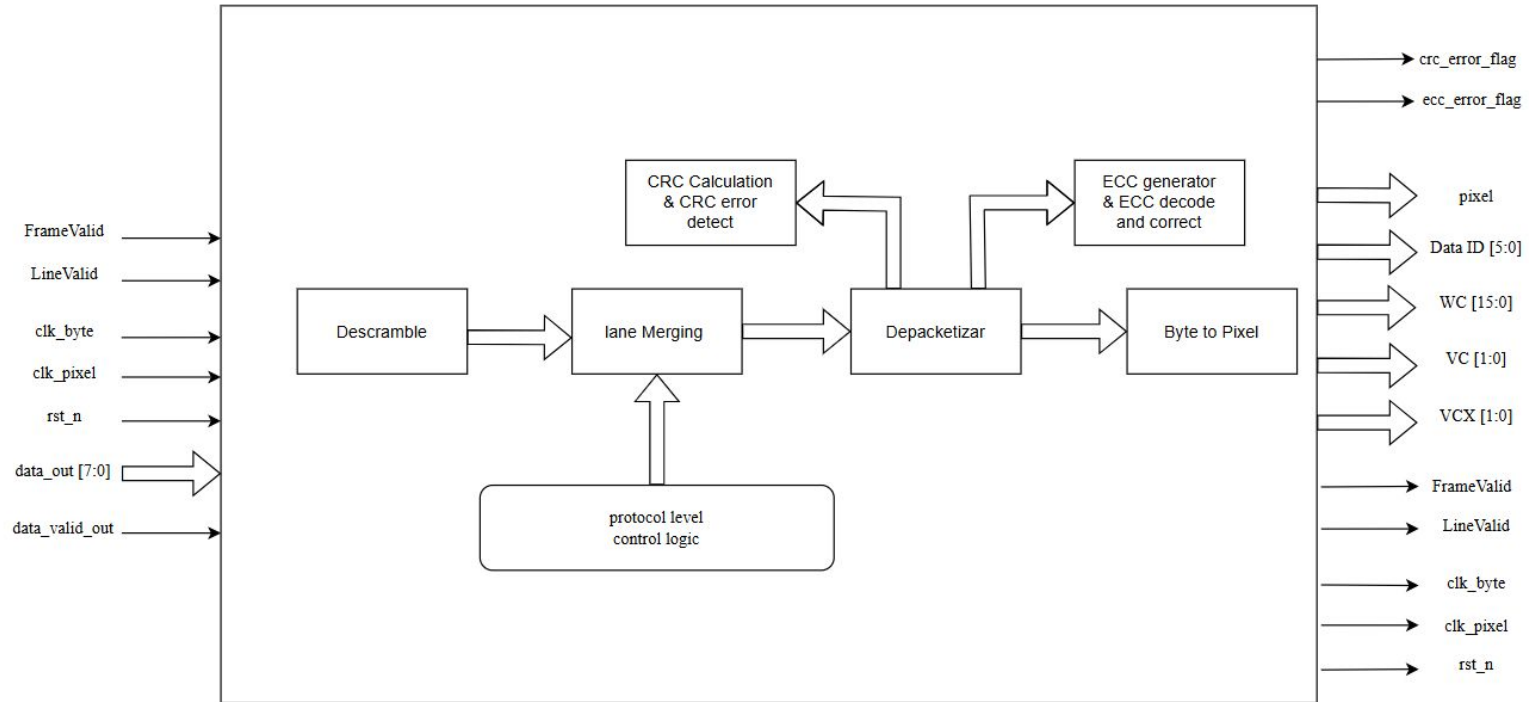
Data_Valid_Out

Indicates whether the scrambled data is valid and ready to be processed by the next block.

When **Data_Valid_Out** is high, the scrambled data is ready for further processing. When it's low, the scrambled data is invalid or not ready.

Receiver

CSI-2 Receiver



Descramble

Descramble



Descrambling (XOR Operation):

The receiver receives the **scrambled data**:

01110110 11001111

PRBS: 10111010 01100101

Descrambled Data (XOR Result): 11001100 10101010 (Original Data)

Descramble



Scrambled_Lane_Data_In

The scrambled data received from the transmitter (scrambled using the same PRBS).

Data_Valid_In

Indicates whether the incoming scrambled data is valid and should be processed.

Packet_Control_In

Provides control signals for the descrambling process, just like in the **scramble** block.

Descramble



Descrambled_Lane_Data_Out

The data after descrambling, which should match the original data before scrambling.

Data_Valid_Out

Indicates whether the descrambled data is valid and ready for further processing.

Descramble



Receive Scrambled Data

The **Descramble** block first receives **scrambled data** from the transmitter, which has already been **scrambled using the PRBS**.

The data comes in through **Parallel_Lane_Data_In**.

Use the Same PRBS

The **Descramble** block needs to **generate the same PRBS** that was used for scrambling. This is where the **initial seed** (used to generate the PRBS at the transmitter) comes into play.

Since both the transmitter and receiver use the same **initial seed**, they can generate the same **PRBS** sequence to **de-scramble** the data.

XOR with PRBS

The block takes the **scrambled data** and **XORs** it with the same **PRBS** that was used for scrambling.

The **XOR operation** will reverse the scrambling process, and the original data will be recovered.

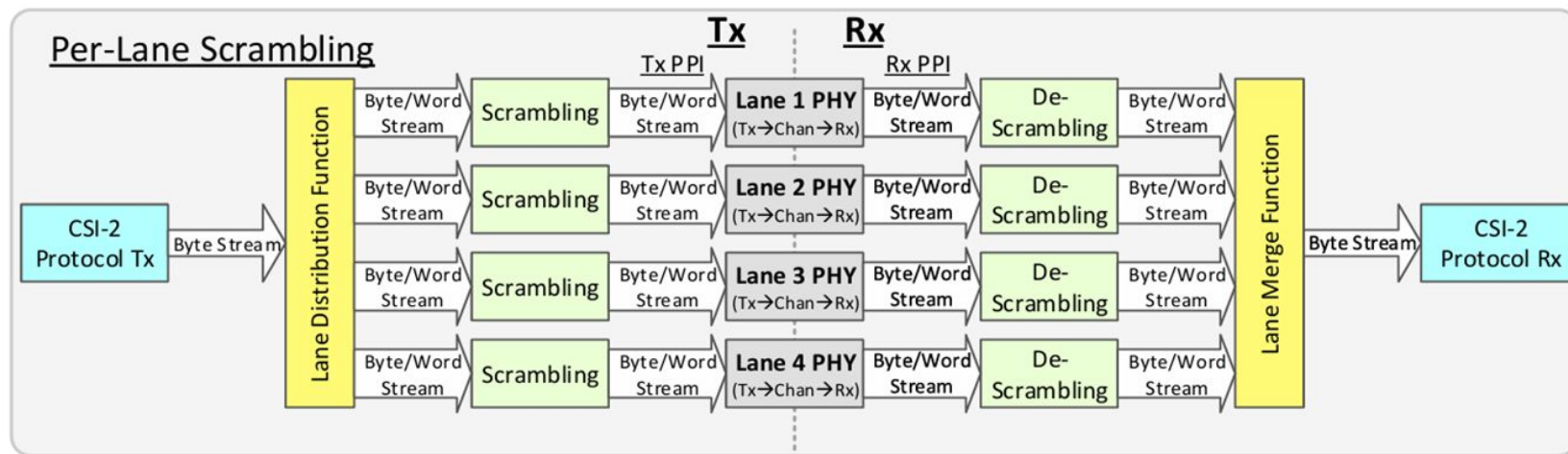
Output the Descrambled Data

Once the data is de-scrambled, it's sent out through **Descrambled_Lane_Data_Out**.

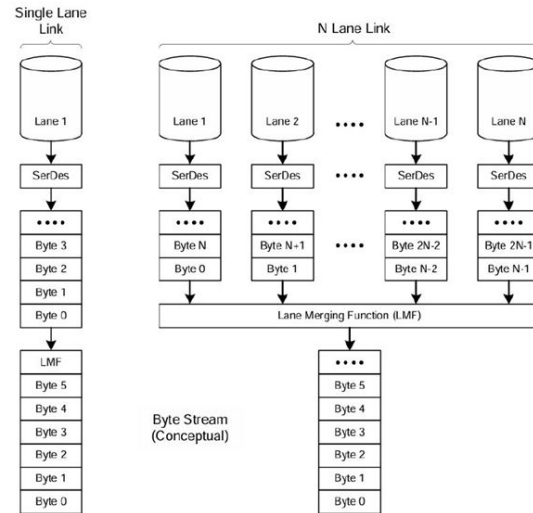
The **Data_Valid_Out** signal is activated to indicate that the descrambled data is valid and ready for further processing.

Lane Management

Lane Management



Lane Management



Lane Management



Descrambled_Lane_Data_In

This is the data that has already been **descrambled** by the **Descramble** block.

The data comes from multiple lanes, where each lane might have part of the data stream.

Data_Valid_In

This signal indicates that the incoming **descrambled data** is **valid** and can be processed.

The **Lane Management** block will only process the data when this signal is high (1), which means the data is correct and ready for handling.

Ready_In

This signal indicates whether the **Lane Management** block is ready to accept new data.

If the **Ready_In** signal is high (1), the system is ready to accept more data from the previous block.

If it is low (0), the system is not ready and will wait for the next valid input.

Lane Management



Packet_Stream_Out

This is the final data output, which contains the processed data ready for transmission or further processing in the system.

The **Lane Management** block organizes the lanes and the data flow, so this output carries the correctly assembled packet stream.

Data_Valid_Out

This signal indicates whether the **Packet_Stream_Out** is valid and can be sent to the next block.

When this signal is high (1), the data on the output is valid and ready for processing.

If it's low (0), the data is invalid or not yet ready to be transmitted.

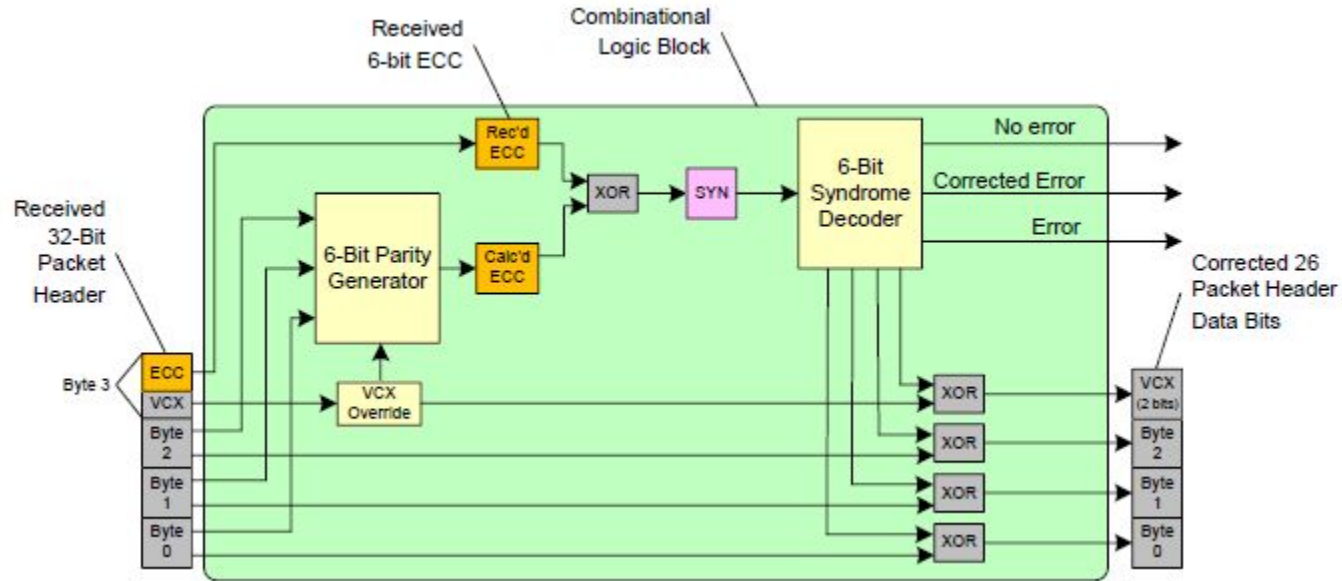
End_Of_Packet_Out

This signal indicates the **end of the packet**. It is typically used to mark the boundary of a complete packet of data.

When this signal is high (1), it means that the current packet is complete, and no more data is expected for the current stream.

ECC

ECC Signal Flow: Receiver (RX) Side - Checking & Correction



ECC Signal Flow: Receiver (RX) Side - Checking & Correction

Function: On the receiver side, the ECC block's function is ECC Checking and Correction. It validates the incoming 32-bit header, corrects single-bit errors, and flags uncorrectable errors.

Input from Lane Management:

- **received_header_in[31:0]**: The full 32-bit packet header, reconstructed from the D-PHY lanes.
- **header_valid_in**: A control signal indicating that received_header_in is valid.
- **clk, rst**: Standard clock and reset signals.

Internal Process (Pipeline):

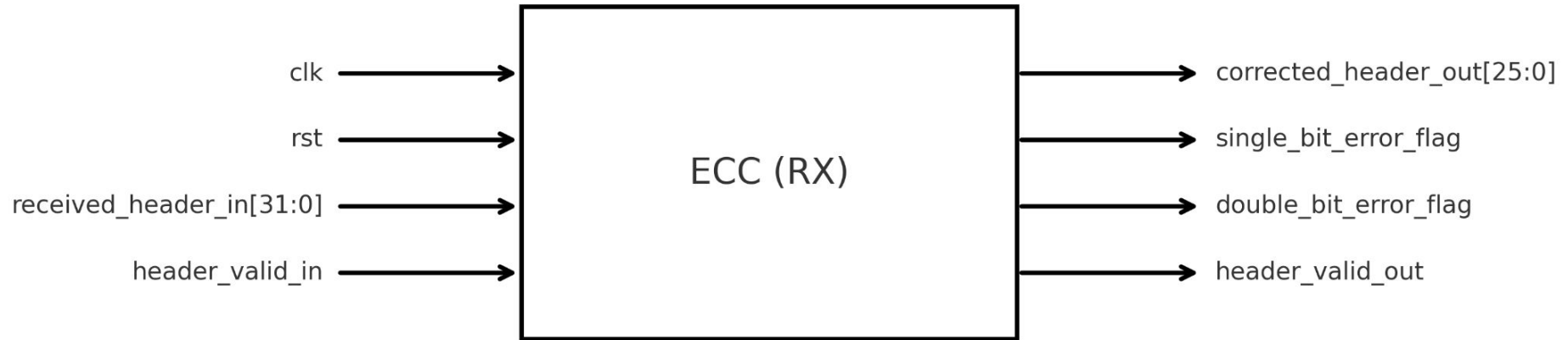
1. Split Header: The 32-bit input is split into **D_rec[25:0] (data)** and **ECC_rec[5:0] (ECC)**.
2. Parity Generation: The D_rec is used to recalculate the ECC, producing **P_calc[5:0]**.
3. Syndrome Calculation: The P_calc is XORed with ECC_rec to generate the 6-bit Syndrome.
4. Syndrome Decoding: The Syndrome is decoded to identify the error type.
5. Correction: If a single-bit error is found, a Correction vector is generated and XORed with D_rec to flip the faulty bit.

Output to Depacketizer & CSRs:

- **corrected_header_out[25:0]**: The 26-bit header data, now verified and corrected. This goes to the Depacketizer.
- **single_bit_error_flag**: Pulses high if a single-bit error was detected and corrected (To CSRs).
- **double_bit_error_flag**: Pulses high if an uncorrectable error was detected (To CSRs).
- **header_valid_out**: Indicates that corrected_header_out is valid.

ECC Signal Flow: Receiver (RX) Side - Checking & Correction

ECC Signal Flow — RX (Check & Correct)



Example: Error Correction at the RX

- **Scenario:** We receive a RAW10 packet header that should have a Word Count of 1600 (0x0640), but a single-bit error occurred in transmission, flipping bit D10 (part of the Word Count).
 1. **Receive Corrupted Header (Input Signal):**
 - The **received_header_in[31:0]** signal arrives at the ECC block. Its data portion (**D_rec**) is now incorrect because D10 is flipped, but its ECC portion (**ECC_rec**) is still the original 0x3A.
 2. **Internal Process: Detect and Correct the Error:**
 - **Recalculate ECC:** The Parity Generator calculates a new ECC based on the corrupted **D_rec**. This **P_calc** will be different from the received **ECC_rec**.
 - **Calculate Syndrome:** The hardware computes **Syndrome = ECC_rec $\oplus$ P_calc**. Since the values are different, the result is a **non-zero Syndrome**. For an error in bit D10, the syndrome will specifically be 0x23.
 - **Decode Syndrome:** The Syndrome Decoder's lookup table maps the value 0x23 to bit position 10.
 - **Correct:** The decoder generates a correction vector with a single '1' at position 10. This is XORed with the corrupted **D_rec**, flipping D10 back to its correct value.
 3. **Final Outputs:**
 - The **corrected_header_out[25:0]** signal now carries the original, correct header data with the Word Count restored to 1600. This is sent to the Depacketizer.
 - The **single_bit_error_flag** signal pulses high for one clock cycle to log the event.
 - The **double_bit_error_flag** remains low.
 - The **header_valid_out** is asserted, confirming the output data is now valid.

ECC Parity Generation Rules

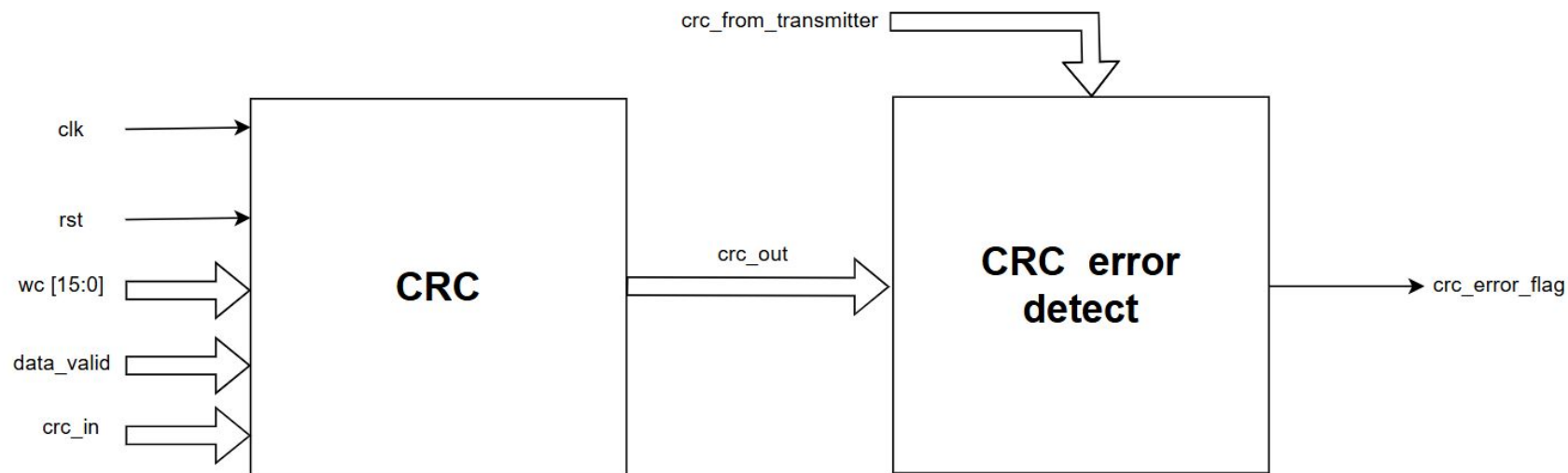
Bit	P7	P6	P5	P4	P3	P2	P1	P0	Hex
0	0	0	0	0	0	1	1	1	0x07
1	0	0	0	0	1	0	1	1	0x0B
2	0	0	0	0	1	1	0	1	0x0D
3	0	0	0	0	1	1	1	0	0x0E
4	0	0	0	1	0	0	1	1	0x13
5	0	0	0	1	0	1	0	1	0x15
6	0	0	0	1	0	1	1	0	0x16
7	0	0	0	1	1	0	0	1	0x19
8	0	0	0	1	1	0	1	0	0x1A
9	0	0	0	1	1	1	0	0	0x1C
10	0	0	1	0	0	0	1	1	0x23
11	0	0	1	0	0	1	0	1	0x25
12	0	0	1	0	0	1	1	0	0x26
13	0	0	1	0	1	0	0	1	0x29
14	0	0	1	0	1	0	1	0	0x2A
15	0	0	1	0	1	1	0	0	0x2C
16	0	0	1	1	0	0	0	1	0x31
17	0	0	1	1	0	0	1	0	0x32
18	0	0	1	1	0	1	0	0	0x34
19	0	0	1	1	1	0	0	0	0x38
20	0	0	0	1	1	1	1	1	0x1F
21	0	0	1	0	1	1	1	1	0x2F
22	0	0	1	1	0	1	1	1	0x37
23	0	0	1	1	1	0	1	1	0x3B
24	0	0	1	1	1	1	0	1	0x3D
25	0	0	1	1	1	1	1	0	0x3E

CRC

Cyclic Redundancy Check

At receiver side:

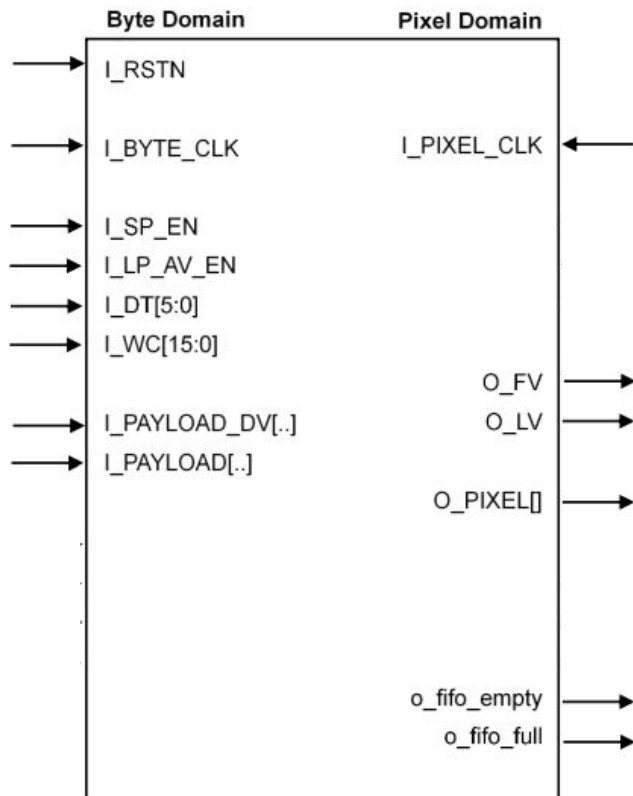
CRC Calculation then error detect



Byte to Pixel

Byte to Pixel converter

At receiver side:



Byte to Pixel converter-Byte Domain

I_BYTE_CLK	Byte-domain clock — the clock at which incoming bytes are received
I_RSTN	Active-low reset for the byte domain logic
I_SP_EN	“Short packet enable” — indicates that a short packet has been received in the byte domain side
I_LP_AV_EN	“Long packet active enable” — indicates that a video / long packet is active in the byte domain.
I_DT[5:0]	Data Type field (6 bits) from the CSI-2 header — tells which kind of data is coming (RAW, RGB, etc.).
I_WC[15:0]	Word Count (16 bits) from the header of the incoming packet — how many payload bytes follow.
I_PAYLOAD[..]	The payload data, and the bit width depends on the lanes and data modes (1:8/1:16)
I_PAYLOAD_DV[..]	The payload data byte valid indicator, active-high; each 8 bits in I_PAYLOAD[..] correspond to 1 bit in I_PAYLOAD_DV[..]

Byte to Pixel converter-Pixel Domain

I_PIXEL_CLK	Pixel domain clock for output side.
O_FV	Frame Valid output in pixel domain.
O_LV	Line Valid output in pixel domain.
O_PIXEL[]	Pixel data output (unpacked) — width depends on the configured pixel format.
o_fifo_full	Internal buffer full Indicator, active-high (I_BYTE_CLK domain).
o_fifo_empty	Internal buffer empty indicator, active-high (I_PIXEL_CLK domain).

Timeline

Scope

RTL	1st Nov.	30th Dec.	8 Weeks
Verification	16th Jan.	13th Mar .	8 Weeks
Synthesis: ● Spyglass ● DFT ● Floorplanning ● Powerplanning ● Placement ● CTS ● Routing	14th Mar.	8th May	8 Weeks

References

References

- [1] MIPI Alliance, "Specification for Camera Serial Interface 2 (CSI-2®) Version 4.0.1," MIPI Alliance, May 23, 2022
- [2] MIPI Alliance, "Specification for D-PHY Version 2.5," MIPI Alliance, Jul. 5, 2019. [Online].
- [3] Gowin MIPI Byte to Pixel Converter IP
|Sep.5,2025:https://www.gowinsemi.com/upload/database_doc/2753/document/6820dee2cd163.pdf
- [4] Intel Corporation, "*MIPI CSI-2 Intel® FPGA IP User Guide*," Doc. No. 813926, Version 1.1.0, Apr. 26, 2024. OnlineOnlineOnline. Available: <https://www.intel.com/content/www/us/en/docs/programmable/813926.html>
- [5] Gowin MIPI Pixel to Byte Converter IP
|Sep.5,2025:https://www.gowinsemi.com/upload/database_doc/2752/document/6820df0ae4095.pdf
- [6]

Thank You!